

RENANCE

FEDERAL UNIVERSITY OF PETROLEUM RESOURCES, EFFURUN

CSC 212 — Past Questions & Materials
by Renance

Course: CSC 212
Institution: Federal University of Petroleum Resources, Effurun
Papers in this pack: 1
Levels: 200

This study pack was compiled and restructured by Renance for CSC 212 students at Federal University of Petroleum Resources, Effurun. It brings together 1 past paper(s)/material(s) into one clean, readable document so you can revise without chasing scattered PDFs. Practise these same questions interactively on Renance with instant answers, AI explanations, and full CBT mock exams — over 235,000+ questions across JAMB, Post-UTME, WAEC, NECO and university courses.

Study online: renance.app
Get the app: Google Play — [dev.renance.renance](https://play.google.com/store/apps/details?id=dev.renance.renance)

Generated by Renance AI

Paper 1 of 1 — CSC 212 FUPRE-: Mobile Application Development Course Handout

Level 200 • First Semester • Handout

FUPRE-CSC-212: Mobile Application Development

Course contents TO COVER

1. Mobile Platforms: Definition, Purpose, Characteristics of and Applications
2. The Google, Android and Apple iOS platform and operating system
3. Building Mobile Apps: Native App, hybrid and Cross-platform apps, Progressive web apps
4. Activities, Intents and Fragments
5. Standard Integrated Development Environment: Android Studio and Xcode
6. App Development/Programming, Testing and Debugging on devices using Emulators/Simulators
7. Building a simple app/game: Design Challenges
8. Services, broadcasts, receivers and data persistence
9. The Mobile Application Development Lifecycle
10. Mobile Application Front-End and Back-End
11. Key Mobile Application Services
12. Platform APIs for user interface, graphics, networking, data and web services
13. Processes, threads and Internet Access
14. Process Improvements: classes, events, properties and methods, threads and runnable(s) for event processing, event listeners and responding to events
15. The activity lifecycle
16. Security Concerns: The Android Security model
17. APK contents
18. Offloading and Outsourcing: Cloud platforms/infrastructure for mobile applications: Infrastructure as Service (IaaS), platform as service (PaaS) and Software as service (SaaS)
19. Challenges and Issues in Mobile Application Development
20. Building smarter devices

LECTURE 1

A mobile application, or app, is a software program designed to run on mobile devices like smartphones and tablets. Apps are different from desktop applications, which run on computers.

How do apps work?

- ☐ Apps are a type of software that perform specific tasks.
- ☐ When an app is opened, it runs within the device's operating system.
- ☐ Apps can access the device's features, like the camera, microphone, and keyboard.
- ☐ Apps can also sync with cloud storage services like OneDrive or Dropbox.

How to create an app?

- ☐ Define the app's idea
- ☐ Choose the app development method

- Create the app
- Test the app on iOS and Android devices
- Submit and publish the app to app stores
- Continuously improve and update the app

Types of apps

- Native apps: Developed specifically for a particular mobile operating system, like iOS or Android
- Progressive web apps (PWAs): A type of app
- Hybrid apps: A type of app

A "mobile platform"

A "mobile platform" refers to the underlying software system on a portable device like a smartphone or tablet, essentially providing the framework for running applications and accessing services, including the operating system, hardware capabilities, and necessary tools for developers to create mobile apps that can function on that device; key characteristics include portability, connectivity, touch-based interfaces, and the ability to utilize various sensors, while applications range from communication tools like messaging apps to productivity software, entertainment platforms, and specialized industry-specific tools depending on the device and user needs.

Key points about mobile platforms:

□ Definition:

A mobile platform is the foundation on which mobile applications run, consisting of the operating system, hardware components, and development tools that enable software to be utilized on a portable device.

□ Purpose:

The primary purpose of a mobile platform is to provide a user-friendly environment for accessing various applications and services on a mobile device, allowing users to interact with information and perform tasks while on the move.

□ Characteristics:

1. Portability: Mobile devices are designed to be easily carried around and used anywhere.
2. Connectivity: Mobile platforms facilitate wireless connections like cellular data, Wi-Fi, and Bluetooth to access the internet and communicate with other devices.
3. Touchscreen interface: Most mobile platforms rely on touch-based interaction for navigation and input.
4. Sensors: Many mobile devices include sensors like GPS, accelerometer, camera, and microphone to enhance app functionality.
5. Battery power: Mobile devices are designed to operate on battery power for extended periods.

□ Major Mobile Platforms:

1. iOS (Apple): Used on iPhones and iPads

2. Android (Google): Used on a wide range of smartphones and tablets from various manufacturers
3. Windows Mobile (Microsoft): Primarily used on Windows Phone devices

Applications of Mobile Platforms:

1. -Communication: Messaging apps (WhatsApp, Telegram), social media platforms (Facebook, Instagram), video calling apps (Zoom, Skype)
2. -Productivity: Email clients, calendars, to-do list apps, document editing tools
3. -Entertainment: Streaming services (Netflix, Spotify), gaming apps (Candy Crush, PUBG)
4. -Navigation: GPS apps (Google Maps, Waze)
5. -Finance: Mobile banking apps, stock trading platforms
6. -Health & Fitness: Fitness trackers, calorie counting apps, medical information platforms
7. -Shopping: Online retail apps, price comparison tools
8. -Business: Customer relationship management (CRM) apps, project management tools, employee collaboration platforms

Mobile platforms are basically those that allow software and services to be run on devices (Fling 2009). Examples of mobile platforms include Palm, BlackBerry, iPhone, Android and Windows Mobile. Mobile operating systems provide tools that allow application to share data and services.

Google Android" and "Apple iOS" are the two dominant mobile operating systems, with Android developed by Google and used on a wide variety of devices, while iOS is developed by Apple and exclusively used on Apple devices like iPhones and iPads; essentially, Android offers more customization and flexibility due to its open-source nature, while iOS is known for its seamless integration and user-friendly interface within Apple's ecosystem.

Key points about Android and iOS: Comparison

□ Developer:

Android is developed by Google, while iOS is developed by Apple.

□ Open vs Closed Source:

Android is open-source, meaning various manufacturers can use it on their devices, while iOS is closed-source and only available on Apple devices.

□ Customization:

Android allows for greater customization with different launchers, widgets, and themes, whereas iOS has a more standardized user interface.

□ App Store:

Android apps are downloaded from the Google Play Store, while iOS apps are downloaded from the Apple App Store.

Introduction:

When it comes to smartphones, the debate between iPhone and Android users has persisted for years. Each platform has its loyal fans and unique features that cater to different needs. top 10 differences between iPhone and Android to help you decide which one might be the

better choice for you.

1. Operating System

The most significant difference lies in their operating systems. iPhones run on Apple's iOS, known for its smooth performance and user-friendly interface. In contrast, Android devices utilize the Android operating system developed by Google, offering more customization options.

2. Customization

Android is celebrated for its high level of customization. Users can modify the home screen layout, install custom launchers, and change the overall look of their devices. On the other hand, iOS maintains a more uniform design, focusing on simplicity and ease of use with limited personalization options.

3. App Store

Each platform boasts its own app store. iPhones access the Apple App Store, while Android devices use the Google Play Store. Although both stores offer a vast selection of apps, the Apple App Store is often viewed as more curated and secure. Android, however, allows users to download apps from third-party sources, providing greater variety.

4. Hardware Variety

Android's open-source nature means multiple manufacturers produce Android devices, resulting in a wide range of choices in hardware, designs, and price points. In contrast, iPhones are exclusively manufactured by Apple, leading to fewer options but ensuring consistent quality and performance.

5. Price Range

Android caters to a broader audience with devices available across various price points, from budget-friendly options to high-end flagship models. iPhones generally come with a higher price tag and fewer budget options, making Android a more accessible choice for many consumers.

6. User Interface

iOS typically offers a straightforward and intuitive user interface, making it easier for newcomers to navigate. While iPhones provide a consistent experience across devices, Android can vary significantly depending on the manufacturer. Some Android devices come with heavy customizations, which can be confusing for some users.

7. Software Updates

Apple is renowned for its timely software updates. When a new iOS version is released, it's available for all compatible devices almost immediately, ensuring users receive the latest features and security updates. Conversely, Android updates can vary by manufacturer and carrier, resulting in delays or even a lack of updates for some devices.

8. Security and Privacy

Apple places a strong emphasis on user privacy and security, with features like Face ID and

end-to-end encryption for iMessages. While Android devices also prioritize security, they have faced more issues with malware and breaches due to the platform's open-source nature. Nevertheless, both platforms offer robust security features, and users should practice safe browsing habits.

9. Integration with Other Devices

For users with multiple Apple products, such as a MacBook or Apple Watch, the integration is seamless. Features like Handoff and AirDrop facilitate easy file sharing and task continuation across devices. While Android integrates well with Google services, it lacks the same level of compatibility with hardware from other manufacturers.

10. Customer Support

Apple is known for its excellent customer support. Users can visit Apple Stores for in-person assistance, making it easier to resolve issues. Android support, however, varies by manufacturer, some offer outstanding support while others may fall short. This discrepancy can be crucial for users who prioritize customer service.

Conclusion:

Choosing between an iPhone and an Android device ultimately comes down to personal preference and priorities. If you value customization, a broader range of hardware options, and budget-friendly choices, Android may be the better option. Conversely, if you prefer a streamlined experience, regular software updates, and strong integration with other devices, an iPhone could be the way to go.

Both platforms have their strengths and weaknesses, so consider which features matter most to you before making a decision. Regardless of your choice, both iPhone and Android offer impressive capabilities to cater to a variety of user needs.

When building mobile apps, you can choose between "native apps" which are built specifically for a single operating system (like iOS or Android), "hybrid apps" which combine web technologies within a native container, "cross-platform apps" that use a single codebase to target multiple platforms, and "progressive web apps" (PWAs) which are essentially advanced web pages that can be installed and behave like native apps, accessible directly through a browser.

Key Differences:

☐ -Native Apps:

☐ Pros: Best performance, full access to device features, seamless user experience.

☐ Cons: Requires separate development for each platform (iOS, Android), longer development time.

☐ Example Frameworks: Xcode (iOS), Android Studio (Android)

☐ -Hybrid Apps:

☐ Pros: Single codebase for multiple platforms, faster development time, easier to maintain.

☐ Cons: May have performance limitations, potentially less access to native device features.

☐ Example Frameworks: Ionic, Cordova

☐ -Cross-Platform Apps:

- Pros: Similar to hybrid apps, single codebase for multiple platforms, faster development.
- Cons: May not achieve the same level of native performance as a truly native app.
- Example Frameworks: React Native, Xamarin
- Progressive Web Apps (PWAs):
- Pros: Works across platforms, can be installed on devices, accessible directly through a browser, can work offline.
- Cons: May have limitations in accessing certain device features compared to native apps, relies on browser capabilities.

Choosing the Right Option:

- For performance-critical apps that need full access to device features: Choose a native app.
- For a fast development cycle and cross-platform compatibility with basic features: Choose a hybrid or cross-platform app.
- For a web-based app with offline functionality and installability: Choose a PWA.

In Android development, "Activities" represent individual screens within an app, "Intents" act as messages to navigate between activities, and "Fragments" are reusable UI components that can be dynamically added within an Activity, allowing for more flexible screen layouts and interactions within a single screen.

Breakdown:

- Activity:
 - Represents a single screen on the device.
 - Has its own lifecycle, including creation, pausing, resuming, and destruction.
 - Used to manage the user interface and interactions on a specific screen.
- Intent:
 - A message that tells Android to perform an action, like launching a new Activity, opening a system component (like the camera), or sharing data between apps.
 - Can carry extra data to be used by the receiving activity.
 - Used for navigation between different Activities.
- Fragment:
 - A modular piece of a user interface that can be embedded within an Activity.
 - Has its own lifecycle, but is managed by the parent Activity.
 - Useful for creating dynamic layouts where parts of the screen can be updated independently.

Key points to remember:

- Relationship:

You use Intents to start new Activities, and you can embed multiple Fragments within a single Activity to create a complex screen.

- Flexibility:

Fragments allow for more dynamic and reusable UI designs compared to Activities alone.

- Data Transfer:

Intents can be used to pass data between Activities, while communication between Fragments and their parent Activity can be achieved through interfaces or other

mechanisms.

When discussing standard Integrated Development Environments (IDEs) for mobile app development, Android Studio is considered the standard for creating Android apps, while Xcode is the standard for developing iOS applications; essentially, Android Studio is used for developing on Google's Android platform, while Xcode is used for Apple's iOS platform.

Key points about each IDE:

□ Android Studio:

Developed by Google, it allows developers to build apps for all Android devices, offering features like a visual layout editor, debugging tools, and integration with Google services.

□ Xcode:

Developed by Apple, it provides a comprehensive suite of tools for designing, developing, and debugging apps for iOS, macOS, iPadOS, tvOS, and watchOS.

Important distinction:

□ Platform compatibility:

□ Android Studio can be used on Windows, macOS, and Linux, while Xcode is only available on macOS.

When developing applications, emulators and simulators are software programs that allow developers to test and debug their apps on virtual devices, mimicking the behavior of real smartphones or tablets, without needing physical hardware, enabling them to quickly iterate through different device configurations and identify potential issues across various screen sizes and operating systems.

Key points about using emulators and simulators for app development:

□ Functionality:

They replicate the hardware and software environment of a specific device, including features like touch input, screen orientation, and device settings, allowing developers to run their app as if it were on a real device.

□ Benefits:

□ Cost-effective: Eliminates the need to purchase multiple physical devices for testing.

□ Rapid iteration: Allows for quick testing and debugging cycles by easily switching between different device configurations.

□ Cross-platform testing: Test an app on various operating systems (like Android and iOS) without needing separate physical devices.

□ Controlled environment: Enables precise manipulation of device settings for targeted testing.

□ Distinction between Emulator and Simulator:

□ -Emulator: Provides a more complete representation of a device, including hardware details like processor and memory, offering a closer simulation to real-world performance.

□ -Simulator: Primarily focuses on the software aspects of a device, mimicking the operating system and application behavior without detailed hardware emulation.

How to use emulators and simulators for testing and debugging:

□ Set up an emulator/simulator:

□ Android Studio: Access the "Android Virtual Device Manager" to create virtual Android

devices with different specifications.

- Xcode: Utilize the iOS Simulator within Xcode to test apps on various iPhone and iPad models.
- Deploy your app:
 - Build your app in the development environment and deploy it to the chosen emulator/simulator.
 - -Perform testing:
 - Basic functionality: Test core app features like navigation, data input, and interactions.
 - Device compatibility: Check if the app behaves as expected on different screen sizes and resolutions.
 - Performance testing: Monitor app responsiveness and loading times under various conditions.
 - -Debugging:
 - Breakpoints: Set breakpoints in your code to pause execution at specific points and inspect variables.
 - Logging: Use logging statements to track the app's state and identify potential issues.
 - Device debugging tools: Utilize platform-specific debugging tools to examine app behavior and diagnose problems.

Limitations of emulators and simulators:

- -Performance discrepancy:

May not perfectly replicate the performance of real devices, especially for hardware-intensive tasks like graphics or battery usage.

- Device-specific features:

Might not accurately simulate all unique features of a specific device, like sensor capabilities or hardware-specific APIs.